TY B.Tech ECE AIML

Academic Session: 2025-26

Project Based Learning Activity

Course Name: Database Management System

Course Code:

Project Title:

Game Library & Recommendation System

Student Name

Aadarsh Mohanan 1032231417

Akshat sharma 1032240204

Department of Electronics and Electrical Engineering

Dr. Vishwanath Karad MITWPU, Pune

Contents:

1. Abstract

- A concise summary of the project (≤250 words).

2. Keywords

- 3–5 keywords related to the project.

3. Introduction

- Problem context, motivation, and scope.

4. Problem Statement

- Clear definition of the issue being solved.

5. Project Objectives

- Goals of the system and expected outcomes.

6. Background & Literature Review

- Related work, existing systems, and limitations.

7. Database Requirement Analysis

- Entities, attributes, functional dependencies.
- Normalization process (1NF → 3NF).

8. Database Schema Design

- ER Diagram.
- Relational schema.
- Primary, foreign, and candidate keys.

9. System Design & Architecture

- **Front-End Design**
  - UI/UX tools and frameworks (e.g., HTML, CSS, JavaScript, React, Angular).
  - Screenshots/wireframes of key pages.
- **Back-End Design**
  - Server-side technology (e.g., Node.js, Django, Flask, PHP).
  - API structure (REST/GraphQL if used).
  - Database connectivity & query handling.
- **Integration**

    - Client-server communication.
    - Middleware, session handling, authentication, etc.

10. Implementation

- SQL queries (CRUD, joins, stored procedures).
- Website functionality (user workflows).
- Screenshots of running system.

11. Testing & Results

- Test cases, input-output.
- Query validation.
- Website testing (functional, UI, performance).

12. Applications

- Real-world use cases.

13. Limitations & Future Scope

- Technical challenges.
- Features to be added later.

14. Conclusion

- Key findings and contributions.

15. Acknowledgment *(optional)*

- Credits to guides, mentors, or contributors.
- Individual Contributions

16. References (IEEE style)

- Journals, books, online resources in IEEE citation style.

1. ABSTRACT

This project, "Game Library & Recommendation System," is a full-stack, data-driven web application designed to serve as a comprehensive platform for video game discovery and management. The system moves beyond a simple CRUD application by implementing a microservice architecture, separating the main web-facing API from a dedicated AI recommendation engine. The core backend is built with Flask and MySQL, providing a robust RESTful API that manages role-based access control (User, Developer, Admin), secure user authentication with password hashing, game management (CRUD), user reviews, and a wishlist system. The frontend, built with HTML, Tailwind CSS, and vanilla JavaScript, offers a dynamic, modal-driven user interface. A key feature is the sophisticated content-based recommendation system, which runs as a separate Flask microservice. This AI engine uses scikit-learn and pandas to build a composite user-taste profile based on highly-rated reviews and wishlisted items, applying feature weighting to game genres and publishers to deliver tailored, accurate recommendations. The platform also includes a complete Developer Dashboard where creators can upload their games, edit them, and view analytics on their performance.

2. KEYWORDS

Full-Stack Application, Microservice Architecture, Flask, MySQL, Recommendation System, Scikit-learn, Pandas, Role-Based Access Control, Tailwind CSS

3. INTRODUCTION

The video game industry has grown into a massive, complex market, making it difficult for users to discover new games that match their specific tastes. Simultaneously, independent developers lack simple, integrated platforms to publish their work and receive direct feedback and analytics. This project addresses these challenges by developing a centralized, dual-purpose platform. The primary motivation is to build a functional, full-stack web application that not only consolidates the principles of database management but also implements advanced, real-world features like a decoupled AI engine, role-based permissions, and a content management dashboard for developers.

4. PROBLEM STATEMENT

The project addresses three core problems:

1. For Users: The lack of a simple, integrated platform to manage game collections, share in-depth reviews, maintain a wishlist, and receive relevant, personalized recommendations that go beyond simple popularity.

2. For Developers: The high barrier to entry for showcasing projects. Developers need a platform where they can upload, manage, and track user engagement (reviews, ratings, wishlists) for their own games.

3. For System Design: Monolithic application design, where a complex, CPU-intensive task like calculating recommendations can slow down the entire user-facing API.

This project aims to provide a streamlined, single-platform solution for these tasks by implementing a microservice architecture with distinct user and developer-facing features.

5. PROJECT OBJECTIVES

- To design and implement a 3NF relational database in MySQL to store comprehensive game, user, review, and wishlist data.
- To develop a robust backend microservice architecture using Flask:
- Main API Server: To manage user authentication (registration with werkzeug.security hashing, login), account management, game/review/wishlist CRUD, and API endpoints for the developer dashboard.
- AI Server: To serve a dedicated /api/recommendations endpoint, encapsulating all AI logic.
- To implement Role-Based Access Control (RBAC) for 'User', 'Developer', and 'Admin' roles, with specific permissions for each.
- To create an intuitive and responsive frontend using HTML, Tailwind CSS, and JavaScript (Fetch API), featuring a modal-based interface for all user interactions.
- To build an advanced content-based recommendation engine using scikit-learn, pandas, and numpy that:
- Generates game vectors using CountVectorizer with feature weighting (e.g., giving higher importance to a game's genre).
- Creates a composite user taste profile by averaging the vectors of games a user has rated highly (>= 4 stars) or added to their wishlist.
- Handles the cold-start problem by recommending popular games to new users.

6. BACKGROUND & LITERATURE REVIEW

This project draws inspiration from popular platforms like Steam, IGN, IMDb, GOG, and itch.io. These systems utilize extensive databases, user-generated content, and developer portals. Our approach simplifies these concepts but validates the core architecture: a robust API, a separate database, and a user-facing client. The use of a decoupled recommendation engine is a standard industry practice to ensure scalability and maintainability. The technology stack (Flask, MySQL, Scikit-learn, Tailwind CSS) aligns with modern, lightweight, and powerful tools for developing data-driven web applications.

7. DATABASE REQUIREMENT ANALYSIS

The system requires a normalized database to manage data integrity and reduce redundancy. The schema is designed in Third Normal Form (3NF).

Entities: The key entities are Users, Games, Genres, Platforms, Reviews, and Wishlist.

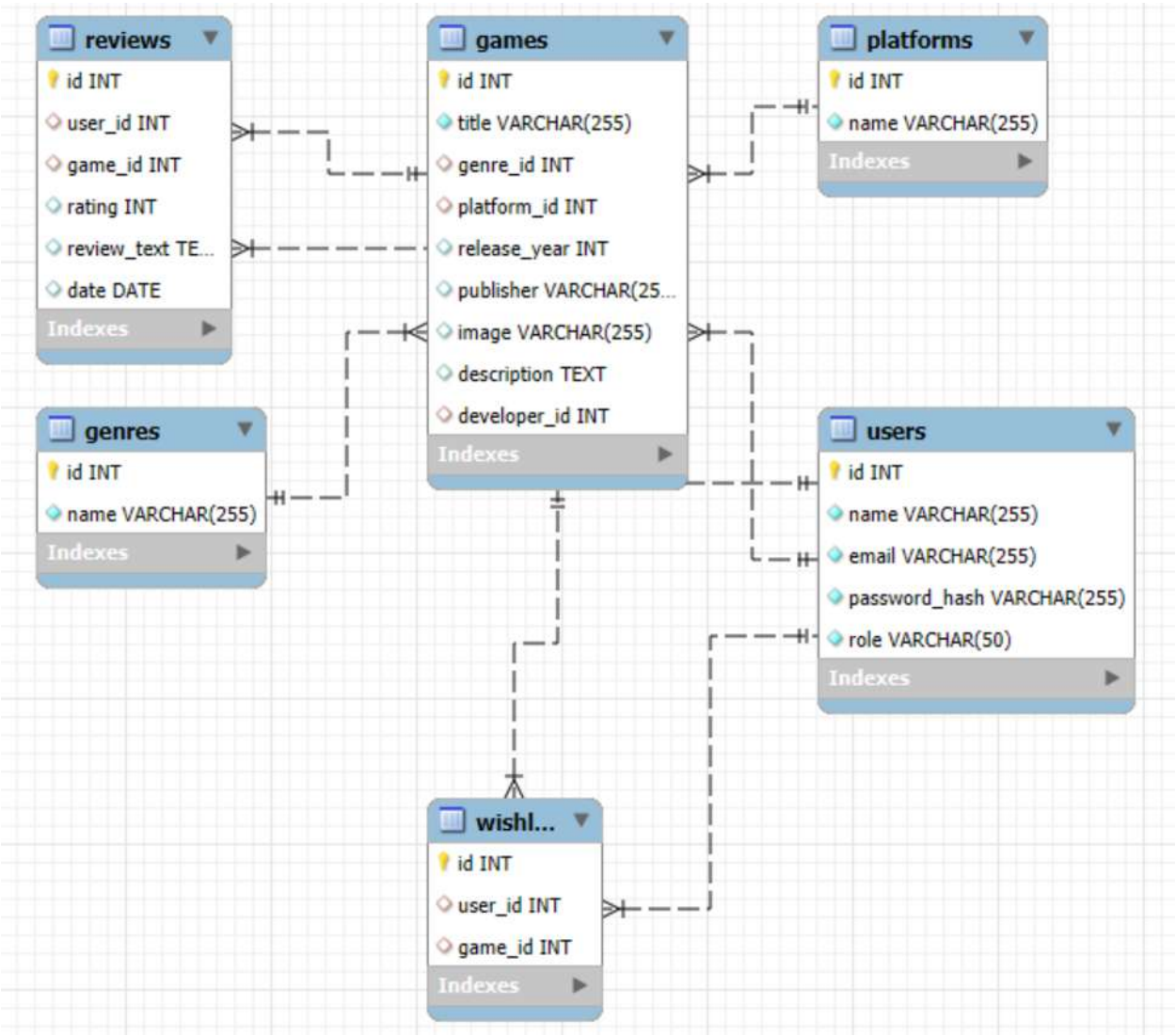
Attributes & Dependencies:

- **Users:** id (PK), name, email (Unique), password_hash (stores hashed password), role (e.g., 'user', 'developer', 'admin').
- **Genres:** id (PK), name (Unique).
- **Platforms:** id (PK), name (Unique).
- **Games:** id (PK), title, release_year, publisher, image, description, genre_id (FK to Genres), platform_id (FK to Platforms), developer_id (FK to Users).
- **Reviews:** id (PK), rating, review_text, date, user_id (FK to Users), game_id (FK to Games). This is a junction table for the M:M relationship between Users and Games (a user can review many games, a game can be reviewed by many users).
- **Wishlist:** id (PK), user_id (FK to Users), game_id (FK to Games). A composite unique key on (user_id, game_id) prevents duplicate entries.

Normalization: The schema is in 3NF. All attributes are dependent on their respective primary keys. Junction tables (Reviews, Wishlist) resolve many-to-many relationships, and transitive dependencies are eliminated (e.g., genre_name is stored in Genres, not in Games).

8. DATABASE SCHEMA DESIGN

A. ER Diagram



B. Relational Schema

- **Users** (`id`, `name`, `email`, `password`)
- **Genres** (`id`, `name`)
- **Platforms** (`id`, `name`)
- **Games** (`id`, `title`, `genre_id`, `platform_id`, `release_year`, `publisher`, `image`, `description`)
- **Reviews** (`id`, `user_id`, `game_id`, `rating`, `review_text`, `date`)
- **Users** (`id` (PK), `name`, `email`, `password_hash`, `role`)

- **Genres** (`id` (PK), `name`)

- **Platforms** (`id` (PK), `name`)

- **Games** (`id` (PK), `title`, `genre_id` (FK), `platform_id` (FK), `release_year`, `publisher`, `image`, `description`, `developer_id` (FK))
- **Reviews** (`id` (PK), `user_id` (FK), `game_id` (FK), `rating`, `review_text`, `date`)
- **Wishlist** (`id` (PK), `user_id` (FK), `game_id` (FK))
-

C. Keys

- **Primary Keys (PK):** `Users(id)`, `Games(id)`, `Genres(id)`, `Platforms(id)`, `Reviews(id)`, `Wishlist(id)`
- **Foreign Keys (FK):**
- `Games(genre_id) references Genres(id)`
- `Games(platform_id) references Platforms(id)`
- `Games(developer_id) references Users(id)`
- `Reviews(user_id) references Users(id)`
- `Reviews(game_id) references Games(id)`
- `Wishlist(user_id) references Users(id)`
- `Wishlist(game_id) references Games(id)`
- **Candidate Keys:** `Users(email)`, `Genres(name)`, `Platforms(name)`, `Wishlist(user_id, game_id)`

9. SYSTEM DESIGN & ARCHITECTURE

A. Front-End Design

The frontend is a single-page application (SPA-like) experience built with:

- **HTML:** For the semantic structure of the site.
- **Tailwind CSS:** A utility-first CSS framework for rapid, responsive, and modern styling.
- **JavaScript (ES6+):** All dynamic functionality is handled using vanilla JavaScript and the `fetch` API for asynchronous communication with the backend.
- **UI/UX:** The interface is modal-driven. User actions like Login, Register, viewing Game Details, leaving Reviews, managing Account Settings, and adding/editing Games all occur in modals, preventing full-page reloads and creating a smooth user experience. It also features dynamic search and multi-select filtering.

B. Back-End Design

The backend is split into two distinct Flask microservices:

1. **Main API Server (`app.py`)**
    - **Technology:** Flask (Python), `mysql-connector-python`.
    - **Responsibility:** Handles all primary business logic and data persistence.
    - **Key Endpoints:**
        - **Authentication:** `/api/register` and `/api/login`. Uses `werkzeug.security` to `generate_password_hash` and `check_password_hash`.
        - **Role Management:** Implements a `get_user_role` helper to check for 'user', 'developer', or 'admin' status.
        - **Account Management:** CRUD endpoints for user accounts (e.g., `/api/user/<id>/change-username`, `/api/user/<id>/convert-to-developer`).
        - **Game/Content API:** `/api/games` (with search and filtering), `/api/genres`, `/api/popular-games` (with sorting).
        - **User Interaction:** `/api/reviews` (POST/GET), `/api/wishlist` (POST/GET/DELETE).
        - **Developer Dashboard:** Endpoints for `POST /api/games`, `PUT /api/games/<id>`, `DELETE /api/games/<id>`, and `/api/developer/<id>/analytics`.
    - **Proxy:** The `/api/recommendations/<id>` endpoint acts as a proxy, forwarding the request to the AI server.
2. **AI Recommendation Server (`ai_server.py`)**
    - **Technology:** Flask (Python), `pandas`, `scikit-learn`, `numpy`.
    - **Responsibility:** Encapsulates all machine learning logic. It runs on a separate port (5001) to avoid blocking the main API.
    - **Logic (`recommendation_system.py`):**

- Loads data from MySQL into `pandas` DataFrames.
- Creates a `tags` feature by combining `genre`, `platform`, and `publisher`, with **genre being weighted 3x** to increase its importance.
- Uses `CountVectorizer` to transform these text tags into a numerical vector space.
- Calculates a `user_profile_vector` by taking the `numpy.mean` of all game vectors the user has wishlisted or rated 4+ stars.
- Calculates the `cosine_similarity` between this user profile vector and all games, returning the top matches.

C. Integration

- **Client -> Main API:** The JavaScript frontend communicates with the Main API (`app.py`) via asynchronous `fetch` requests, sending and receiving data in JSON format.

- **Main API -> AI API:** When a user requests recommendations, `app.py` uses the `requests` library to make a server-to-server HTTP request to `ai_server.py`. This isolates the client from the AI logic and protects the AI endpoint.

10. IMPLEMENTATION

A. SQL & Database

- All database interactions are performed in `app.py` using parameterized SQL queries via the `mysql-connector-python` library to prevent SQL injection.

- Complex `JOIN` statements are used to retrieve game data with its corresponding genre and platform names in a single query.

- `GROUP BY` and aggregate functions (`COUNT`, `AVG`) are used in the `/api/popular-games` and `/api/developer/<id>/analytics` endpoints.

B. Backend (Flask)

- **Authentication:** User registration securely hashes passwords using `werkzeug.security.generate_password_hash`. The login endpoint validates credentials using `check_password_hash`.

- **Authorization:** Most developer and admin endpoints use the `get_user_role` helper function to check permissions before executing any logic (e.g., ensuring only the `developer_id` matching the game or an 'admin' can edit or delete a game).

- **API Design:** The API follows RESTful principles, using appropriate HTTP methods (GET, POST, PUT, DELETE) for different actions.

C. AI System (`recommendation_system.py`)

- **Data Preparation:** `pandas` is used to load data from SQL and clean it (e.g., `fillna('')`). The weighted `tags` feature is created using simple string operations.

- **Model Building:** `CountVectorizer(max_features=5000, stop_words='english')` creates a sparse matrix of token counts.

- **Similarity:** `cosine_similarity` from `sklearn.metrics.pairwise` is used for its efficiency in calculating similarity between high-dimensional vectors.

- **Profile Generation:** The user's taste profile is a single vector, created by averaging the vectors of their liked/wishlisted games (`np.mean(..., axis=0)`). This composite vector represents the user's ideal game.

D. Website Functionality (JavaScript)

- **State Management:** A global `loggedInUser` object in JavaScript holds the user's state (ID, name, role) after login, which is used to conditionally render UI elements (e.g., show "Logout" vs. "Login", show the "Developer Dashboard").

- **Dynamic Rendering:** Functions like `renderGames` and `renderAnalytics` dynamically create HTML elements from JSON data, updating the grid without a page refresh.

- **Event-Driven:** The site uses `addEventListener` for all interactions, from modal triggers to form submissions and dynamic filtering.

11. TESTING & RESULTS

A. Test Cases (Input/Output):

- **Input:** User registers with "test@example.com".

- **Output:** Database shows a new user with a hashed password. `authMessage` shows "Registration successful".

- **Input:** User logs in with "test@example.com".

- **Output:** `loggedInUser` object is populated. "Welcome" message and "Account" button appear. "Developer Dashboard" is hidden.

- **Input:** User clicks "Upgrade to Developer Account".

- **Output:** `Users` table `role` column for that user is updated to 'developer'. "Developer Dashboard" becomes visible.

- **Input:** User (Developer) fills and submits the "Add New Game" form.

- **Output:** A new record is created in the `Games` table with the user's `developer_id`. The game appears in the "My Games" tab.

- **Input:** User searches for "Cyberpunk".

- **Output:** The main game grid filters to show only "Cyberpunk 2077".

- **Input:** User filters by "RPG" genre.

- **Output:** The grid updates to show only games with `genre_id` matching "RPG".

- **Input:** User adds 3 'RPG' games to their wishlist.

- **Output:** The `/api/recommendations` endpoint returns other 'RPG' games (like "The Witcher 3") at the top of the list.

B. Query Validation: API endpoints were tested using Postman to ensure correct JSON responses, error codes (400, 401, 403, 404, 500), and data integrity from backend queries.

C. Website Testing: The website was tested for functionality and UI responsiveness on various screen sizes to ensure a consistent user experience.

12. APPLICATIONS

The Game Library & Recommendation System has several real-world applications:

- **Educational Tool:** Serves as a complete, self-contained project for students learning full-stack development, database design, microservice architecture, and applied machine learning.

- **Independent Developer Platform:** Can be deployed as a live platform for indie developers to host, share, and get feedback on their projects.

- **Community Platform:** Can be scaled into a larger community-driven review site for niche game genres.

13. LIMITATIONS & FUTURE SCOPE

- **Authentication:** The current implementation manages user state on the client side after login. A more secure, production-grade approach would be to implement **session-based authentication or JSON Web Tokens (JWT)** to validate requests on the server.

- **Recommendation Engine:** The current system is purely **content-based**. It cannot capture nuanced preferences that **collaborative filtering** (recommending items that similar users liked) would provide.

- **Scalability:** The AI server re-trains (loads all data) on initialization. For a large-scale application, this model would need to be re-trained offline periodically, and the recommendation logic might be better served by a dedicated ML pipeline.

- **Future Scope:**
    - Implement **Collaborative Filtering** (or a hybrid approach) for more advanced recommendations.
    - Implement **JWT-based authentication** for a secure, stateless API.
    - Build out full **User Profile Pages** where users can see their reviews, wishlisted items, and favorite genres.
    - Allow users to **edit or delete their own reviews**.
    - Expand the admin dashboard for user and content management.

14. CONCLUSION

This project successfully demonstrates the design and implementation of a sophisticated, full-stack web application. By adopting a **microservice architecture**, it cleanly separates the primary business logic from the complex AI-driven recommendation engine. The project effectively integrates a **MySQL** database, a **Flask** backend with **role-based access control**, and a dynamic **Tailwind CSS/JavaScript** frontend. The resulting platform not only serves as a functional game library but also provides a complete, end-to-end ecosystem for both users and game developers.

15. ACKNOWLEDGEMENT

We are mutually thankful for our strong partnership and dedicated collaboration as a team (Aadarsh Mohanan & Akshat Sharma). Our combined efforts were essential in designing, developing, and successfully implementing this project's complex, multi-layered architecture.

Finally, we acknowledge the global open-source community for developing the powerful tools such as Python, Flask, MySQL, Scikit-learn, and Pandas that formed the technical foundation of

our application and made this project possible.

Individual Contributions & Project Ownership

This project was a highly collaborative partnership with a 50/50 division of work. We partitioned the architecture by domain, allowing each member to take full ownership of critical components, ensuring a robust and successful final product.

Aadarsh Mohanan (Lead Frontend & System Architect)

- **Full-Stack Frontend Development (`index.html`):**
    - Architected the complete user-facing application, designing the UI/UX with **HTML and Tailwind CSS**.
    - Engineered all client-side **Vanilla JavaScript** for a dynamic, modal-driven SPA experience, including all API fetch logic.
- **Core API & Security (`app.py`):**
    - Developed the foundational Flask server structure.
    - Engineered the critical security and user management layer, implementing secure **User Authentication** (with `werkzeug`) and the **Role-Based Access Control (RBAC)** system.
- **Microservice Integration:**
    - Masterminded the system's integration, implementing the **API proxy** in `app.py` to securely decouple the main server from the AI microservice.
    - Ensured seamless, end-to-end communication between the client, main API, and AI service.

Akshat Sharma (Lead Data & AI Engineer)

- **Database Architecture (`game_library.sql`):**
    - Designed and normalized the project's data foundation, creating the complete **3NF relational schema** in MySQL.
    - Authored all SQL scripts for table creation, constraints, relationships, and data seeding.
- **Data API & Analytics (`app.py`):**
    - Developed the core data-facing endpoints of the API.
    - Implemented all logic for game, review, and wishlist **CRUD operations** and built the backend API for the **Developer Dashboard** and analytics reporting.

- **AI Recommendation Engine (`ai_server.py`, `recommendation_system.py`):**
    - Engineered the dedicated **Flask microservice** to host the AI model, ensuring the main API's performance was not impacted.
    - Implemented the complete ML pipeline using **Scikit-learn, Pandas, and Numpy**, including feature weighting, composite user-profile generation, and cold-start logic.

16. REFERENCES

[1] P. Py, "Flask web framework," The Pallets Project, Flask Documentation. [Online]. Available: https://flask.palletsprojects.com/. (accessed: September, 2025).

[2] Oracle, "MySQL documentation," Oracle Documentation. [Online]. Available: https://dev.mysql.com/doc/. (accessed: September, 2025).

[3] Mozilla, "HTML: HyperText Markup Language," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. (accessed: September, 2025).

[4] The pandas development team, "pandas-dev/pandas: Pandas," Zenodo, 2020. [Online]. Available: https://pandas.pydata.org/about/citing.html. (accessed: September, 2025).

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020. [Online]. DOI: 10.1038/s41586-020-2649-2.

[7] G. van Rossum, "Python tutorial, Technical Report CS-R9526," Centrum voor Wiskunde en Informatica (CWI), Amsterdam, May 1995.

[8] Tailwind Labs, "Installation," Tailwind CSS Documentation. [Online]. Available: https://tailwindcss.com/docs/installation. (accessed: September, 2025).

[9] The Pallets Project, "Werkzeug: The Python WSGI Utility Library," Werkzeug Documentation. [Online]. Available: https://werkzeug.palletsprojects.com/. (accessed: September, 2025).